Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Olasılık tabanlı yöntemlerden sözlük tabanlı yöntemlere geçiş
  - Şimdiye kadar görülen yöntemler verinin istatistiğini (sembol olasılıklarını) kullanıyordu; model ne kadar iyi kurulursa sıkıştırma o kadar verimli oluyordu.
  - Sözlük tabanlı yöntemlerde sembol olasılığı kullanılmaz, değişken ya da sabit uzunluklu kodlar da kullanılmaz. Sözlükte tutulan şeyler anlamlı kelimeler değil, bir, iki, üç ya da daha fazla karakterden oluşan, anlamlı ya da anlamsız pattern'lardır (ekrandan, 00:20:00; "Sözlük Tabanlı Yöntemler, Bölüm 6").
  - Sözlük statik (hiç değişmez) ya da dinamik (yer dolunca az kullanılan pattern'lar silinip yenileri eklenebilir) olabilir.
- Statik sözlük
  - En basit örnek İngilizce dokümanlar için kurulmuş, yaklaşık yarım milyon kelimelik bir sözlüktür. Okunan kelime sözlükte varsa sıra numarası, yoksa kelimenin kendisi (ASCII) yazılır.
  - Sondan eklemeli dillerde (Türkçe gibi) kelime biçimi sayısı çok büyüyeceği için uygun değildir.
  - $2^{19} = 524\,288$ olduğu için indeks 19 bittir. Başa bir bayrak (flag) biti eklenir: kelime sözlükte varsa $1 + 19 = 20$ bit; yoksa 1 bit bayrak + 7 bit karakter sayısı + 5 karakter × 8 bit $= 48$ bit (ekrandan, 00:24:00).
- String uzunluğu ve sembol başına kod uzunluğu
  - a_1 ($P = 0{,}8$) ve a_2 ($P = 0{,}2$) alfabesinde tek sembol, ikili ve üçlü string'ler Huffman ile kodlanıp karşılaştırılır (ekrandan, 00:35:30 ve 00:40:00).
  - Sembol başına ortalama kod uzunluğu: tek sembolde 1 bit, ikilide $1{,}56/2 = 0{,}78$, üçlüde $2{,}184/3 = 0{,}728$ bit. String uzadıkça sembol başına bit azalır; sözlük yöntemlerinin ana fikri en uzun pattern'ı kodlamaktır.
- LZ ailesi
  - Jacob Ziv ve Abraham Lempel LZ77'yi (1977) ve bir yıl sonra LZ78'i geliştirdi. Diğerleri (LZSS, LZFG, LZRW, LZW, LZMW, LZAP, LZY, LZP) bunlardan türetilmiştir; çoğunlukla buffer boyu, arama biçimi (doğrusal ya da dairesel), sözlük boyu ve statik/dinamik olması gibi parametrelerle oynarlar.
- LZ77 (kayan pencere, sliding window)
  - Pencerenin solu arama alanı (search buffer, sözlük gibi kullanılan, önceden kodlanmış veri), sağı kaynak kod alanıdır (look-ahead buffer). Arama alanı pratikte binlerce byte, kaynak kod alanı onlarca byte olur.
  - Token üç alanlıdır: (offset, eşleşen sembol sayısı, eşleşmeden sonra gelen sembol).
  - Offset alanı $\lceil \log_2 S \rceil$, uzunluk alanı $\lceil \log_2 (L-1) \rceil$, sembol alanı 8 bittir; yaklaşık bir token $11 + 5 + 8 = 24$ bittir (ekrandan, 01:00:00).
  - Arama alanı doğrusal ya da dairesel kuyruk (circular queue, start ve end pointer) olarak tutulabilir (ekrandan, 01:10:00).
- LZ78
  - Kayan pencere yoktur; önceden karşılaşılan string'lerden oluşan, boş başlayan ve boyu hafızayla sınırlı bir sözlük kullanılır.
  - Token iki alanlıdır: (sözlükteki sıra belirleyicisi, sembol) (ekrandan, 01:30:00).
  - Sözlük için en iyi veri yapısı trie ağacıdır. Sözlük dolunca: sözlük dondurulur (FROZEN), ağaç silinip baştan başlanır ya da en az kullanılan pattern çıkarılıp yerine yenisi konur (ekrandan, 01:42:30).

- LZW (Lempel-Ziv-Welch)
  - Sözlüğe codebook denir. Codebook alfabenin sembolleriyle başlar; çıkışa yalnız sözlük indeksi yazılır, sembol yazılmaz.
  - Codebook boyu sınırlıysa sözlük dolunca yeni pattern eklenmez, yalnız var olanlar kullanılır.
- Görüntü sıkıştırma
  - Görüntüye özel bir sıkıştırma yöntemi yoktur; ön işlem (renk azaltma, kümeleme, nicemleme) yapılıp metinde görülen yöntemler uygulanır.
  - GIF ve TIFF içinde LZW, ZIP/GZIP içinde LZ77 türevi ile Huffman, PNG içinde zlib/deflate vardır.
  - JPEG adımları: RGB→YUV (isteğe bağlı), örneklemeyi azaltma, 8×8 bloklar, DCT, nicemleme (Q matrisleri), zigzag tarama, AC'ye RLE, DC'ye DPCM, ardından entropi kodlaması (ekrandan, 02:40:20).

Hocanın Özellikle Vurguladığı Kısımlar

- Tek karakter yerine olabildiğince uzun bir pattern kodlanırsa sıkıştırma başarısı artar
  - Huffman'da karakter yerine hece kodlamak bu yüzden daha başarılıdır. Sözlük yöntemlerinin bütün mantığı buna dayanır.
- Statik sözlük ancak özel durumlarda tercih edilir
  - Belirli bir alanda, sınırlı sayıda kelimeden oluşan dokümanlarda işe yarar. Genelde dinamik yöntemler daha iyi sonuç verir.
- LZ77'de token aynı uzunlukta birden fazla eşleşme varsa slayttaki örnekte en son bulunan (sağdan sola taramada en uzaktaki) eşleşmeyi gösterir: (16, 3, "e")
- Kod çözücü kodlayıcıdan daha basittir
  - Token'ı okur, offset kadar geri gider, uzunluk kadar sembolü kopyalar ve üçüncü alandaki sembolü ekler.
- LZ türevleri arasındaki fark parametrelerdedir
  - Arama alanının ve kaynak kod alanının boyu, offset/length alanlarının uzunluğu, doğrusal ya da dairesel arama, sözlüğün sabit ya da değişken boyu.
- LZW, LZ'ler içinde en başarılı olanıdır ve birçok formatın (GIF, TIFF) içinde kullanılır.

Kısa Tekrar Notları

- Statik sözlük örneği: sözlükte varsa 20 bit, yoksa 48 bit. Sıkıştırma için $20P+48(1-P) < 40$, yani $28P > 8$, $P > 0{,}29$ olmalıdır (ekrandan, 00:30:00).
- Hocanın kullandığı "varyans": olasılıkların ortalamadan mutlak farklarının toplamı. Tek sembolde 0,6; ikilide 0,78; üçlüde 0,792.
- LZ77 token: (offset, length, sonraki sembol). Eşleşme yoksa (0, 0, sembol).
- "sir sid eastman …" LZ77: (0,0,"s"), (0,0,"i"), (0,0,"r"), (0,0,"␣"), (4,2,"d"), (4,1,"e") (ekrandan, 01:00:00).
- LZ78 token: (sözlükteki sıra, sembol). Sözlük 0 = null ile başlar.
- LZW: codebook alfabeyle başlar; çıkışa yalnız indeks yazılır.
- abbbaacbbabbaacacaabb, alfabe {a, b, c}, sınırsız codebook: çıkış 0 1 4 0 0 2 5 5 7 11 3 1.
- JPEG: YUV → downsampling → 8×8 → DCT → nicemleme → zigzag → RLE/DPCM → entropi kodlaması. Kayıp, nicemlemedeki bölme ve yuvarlamadan gelir.
- $Y = 0{,}299R + 0{,}587G + 0{,}114B$.

Detaylı Açıklamalar

- **Statik sözlük ve başabaş noktası.** Sözlükte yarım milyon kelime varsa her kelimenin indeksi 19 bitle yazılır ($2^{19} = 524\,288$). Çözücünün okuduğu bitlerin indeks mi, ham kelime mi olduğunu ayırabilmesi için başa bir bayrak biti konur. Bayrak 0 ise ardından gelen 19 bit indekstir (toplam 20 bit). Bayrak 1 ise ardından gelen 7 bit kelimenin karakter sayısını, sonraki byte'lar karakterleri verir; 5 karakterli kelime için $1 + 7 + 40 = 48$ bit (ekrandan, 00:24:00). Hiç sıkıştırma yapılmasaydı 5 karakterlik kelime $5 \times 8 = 40$ bit tutardı. Kelimenin sözlükte bulunma olasılığı P ise $N[20P + 48(1-P)] = 40N$ eşitliğinden $28P = 8$,

$P \approx 0{,}29$ çıkar. Yani kelimelerin en az yaklaşık %29'u sözlükte olmalıdır; bunun üzerinde sıkıştırma kazancı başlar (ekrandan, 00:30:00).
  - Ders sonrası not: Hoca bir yerde "sıfırsa buradan sonra okuyacağın 20 tane bit" dedi (00:23:17); slayta göre bayraktan sonra 19 bit indeks okunur, 20 bit toplam uzunluktur.
- **Neden uzun pattern?** İki sembollü alfabede ($0{,}8$ ve $0{,}2$) tek sembol kodlanırsa her sembol 1 bit alır. İkili string'ler kodlanırsa olasılıklar 0,64; 0,16; 0,16; 0,04, Huffman kodları 0, 11, 100, 101 olur ve ortalama $1(0{,}64) + 2(0{,}16) + 3(0{,}16) + 3(0{,}04) = 1{,}56$ bit/string, yani 0,78 bit/sembol çıkar (ekrandan, 00:35:30). Üçlü string'lerde kod uzunlukları 1, 3, 3, 3, 5, 5, 5, 5 olur ve $1(0{,}512) + 3 \cdot 3(0{,}128) + 3 \cdot 5(0{,}032) + 5(0{,}008) = 2{,}184$ bit/string, yani 0,728 bit/sembol (ekrandan, 00:40:00). Bu hesaplar python ile doğrulandı.
  - Ders sonrası not: Hoca anlatırken tek sembol durumundaki 0,6 değerini (varyans) ortalama kod uzunluğuyla karıştırdı (00:35:58, "bir öncekinde bu 0,6 idi"). Tablodaki ortalama kod uzunluğu tek sembolde 1 bittir; 0,6 varyans sütunudur.
- **LZ77'nin işleyişi.** Kaynak kod alanının ilk sembolü arama alanında sağdan sola aranır. Bulunduğu her yerde eşleşme ikinci, üçüncü sembolle uzatılmaya çalışılır. En uzun eşleşme bulununca token yazılır, eşleşen semboller ve token'daki sembol arama alanına kaydırılır. Arama alanı dolunca en eski semboller dışarı atılır. "sir sid eastman easily t | eases sea sick seals" örneğinde "e" sağdan 8. ve 16. konumda bulunur, "eas" her ikisinde de eşleşir, "ease" hiçbirinde eşleşmez; token (16, 3, "e") olur (ekrandan, 00:50:00).
  - Ders sonrası not: Hoca pencereyi "stack, ilk giren ilk çıkar" diye tarif etti (00:44:20). İlk giren ilk çıkar (FIFO) davranışı yığına (stack) değil kuyruğa (queue) aittir.
- **LZ77 alan uzunlukları.** Arama alanı uzunluğu S ise offset $\lceil \log_2 S \rceil$ bit tutar; derste $S = 32$ için 5 bit örneği verildi. Pratikte offset 10-12 bit olur. Kaynak kod alanı uzunluğu L ise uzunluk alanı $\lceil \log_2(L-1) \rceil$ bittir. Sembol alanı 8 bittir; bir token yaklaşık $11+5+8 = 24$ bit (ekrandan, 01:00:00). Uzun bir eşleşme (örneğin 16 karakter) 128 bit yerine yaklaşık 30 bitle gönderilebilir.
- **Doğrusal ve dairesel pencere.** Doğrusal pencerede yeni sembol gelince bütün semboller bir kayar ve en eski sembol atılır. Dairesel kuyrukta semboller yerinde durur; start ve end pointer'ları ilerler, yeni sembol en eski sembolün yerine yazılır (ekrandan, 01:10:00). Başarı değişmez ama kaydırma olmadığı için hız kazanılır. Pencere uzunsa dairesel yapı, birkaç bitlikse doğrusal yapı mantıklıdır.
- **LZ78'in işleyişi.** Gelen sembol w sözlükte yoksa sözlüğün sonuna eklenir ve (0, w) yazılır. Varsa bir sonraki sembol k eklenip wk aranır; en uzun bulunan string'in sırası ve onu izleyen sembol token olur, wk sözlüğe eklenir. "sir sid eastman easily teases …" için sözlük: 1 s, 2 i, 3 r, 4 ␣, 5 si, 6 d, 7 ␣e, 8 a, 9 st, 10 m, 11 an, 12 ␣ea, 13 sil, 14 y, 15 ␣t (ekrandan, 01:40:00; 15. satır derste birlikte bulundu). Hızlı arama için trie ağacı kullanılır: kökten alfabe boyu kadar dal çıkar, her düğümün altında o pattern'ı izleyen karakterler bulunur.
- **LZW'nin işleyişi.** Codebook alfabedeki sembollerle başlar. En uzun eşleşen string bulunur, indeksi çıkışa yazılır, string ile ardından gelen sembol codebook'a eklenir, işlem o sembolden devam eder. "abbbaacbbabbaacacaabb" ({a, b, c}) için sınırsız codebook'ta eklenenler: 3 ab, 4 bb, 5 bba, 6 aa, 7 ac, 8 cb, 9 bbab, 10 bbaa, 11 aca, 12 acaa; çıkış 0 1 4 0 0 2 5 5 7 11 3 1 (ekrandan, 02:10:00; python ile doğrulandı). Codebook 8 ile sınırlanırsa 0-7 dolduktan sonra ekleme yapılmaz ve çıkış 0 1 4 0 0 2 5 5 7 7 6 4 olur.
  - Ders sonrası not: Hoca her iki çözümün son adımlarında kalan diziyi "abb" diye okudu (01:56:40 ve 02:02:13). Python ile yapılan doğrulamaya göre sınırsız codebook'ta son iki çıkış 3 (ab) ve 1 (b), sınırlı codebook'ta ise 6 (aa) ve 4 (bb) olmalıdır.
- **Formatlar.** GIF büyük olasılıkla 1987'de CompuServe tarafından geliştirildi ve LZW kullanır; 256 renkle sınırlı olduğu için gri tonlu görüntülerde daha başarılıdır, LZW'nin lisans sorunu yüzünden ticari kullanımı azdır. TIFF de LZW kullanır; sonraki sürümüne JPEG sıkıştırması eklenmiştir. PNG açık bir formattır; 48 bit gerçek renge ve 16 bit gri tona kadar destek verir, içinde zlib/deflate vardır. ZIP ve GZIP'te LZ77 türevi kullanılır: 32 KB'lık kayan sözlük ve 258 byte'lık kaynak kod alanı.
- **JPEG.** İnsan gözü parlaklığa renkten daha duyarlıdır; bu yüzden RGB, YUV'a çevrilip ($Y = 0{,}299R + 0{,}587G + 0{,}114B$) renk bileşenlerinde örnekleme azaltılır. Görüntü 8×8 bloklara bölünür, piksellerden 128 çıkarılarak değerler $[-128, 127]$ aralığına taşınır (T matrisi) ve her bloğa DCT uygulanır (ekrandan, 02:46:40). 64 katsayının ilki DC, diğerleri AC'dir; düşük frekanslar sol üst, yüksek frekanslar sağ alt köşededir. DCT katsayıları

seçilen kalite oranına ait Q matrisine bölünüp yuvarlanır; kayıp buradan gelir. Ardından zigzag tarama, AC katsayılarına RLE, DC katsayılarına DPCM ve entropi kodlaması uygulanır. Açarken her adımın tersi yapılır.
  - Ders sonrası not: Hoca sözlü anlatımda ilk katsayıya "AC", diğerlerine "DC" dedi ve sol üst köşeyi yüksek frekans olarak söyledi (02:44:29, 02:47:19). Slayta göre ilk katsayı DC'dir, düşük frekanslar sol üst köşededir. Hoca bir yerde "düşük frekansları resimden çıkarmaya çalışıyorum" dedi (02:34:17); atılan, gözün algılayamadığı yüksek frekanslardır.
  - Ders sonrası not: Slayttaki $D = M^T T M$ formülü, slayttaki M tanımıyla (i satırı frekans) python ile denendiğinde doğru DCT'yi vermedi; bu tanımla doğru çarpım $D = MTM^T$'dir.
  - Ders sonrası not: Hoca JPEG 2000'in "tamamıyla kayıpsız" olduğunu söyledi; JPEG 2000 büyük olasılıkla hem kayıplı hem kayıpsız kipi destekler (wavelet dönüşümü kullanır).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.